

Experiment -1

Linux Programming Environment & Basic Commands

A) Setting up of Linux Programming Environment

We can do C programming online or offline according to the needs. It is required for us to keep working on a Linux Environment.

Online:

Cocalc.ai: It is an online platform that provides a Linux-based environment through a web browser. It allows us to write, compile, and execute C programs without installing Linux on our computer.

Steps to use CoCalc for C Programming

1. Open the CoCalc website in a web browser.
2. Create an account or log in to an existing account.
3. Create a new project.
4. Open the Linux Terminal from the project.
5. Create a C program file using an editor.
6. Write the required C program and save the file.
7. Compile the program using the GCC compiler
8. Execute the compiled program
9. The output of the program will be displayed in the terminal.

Offline:

Methods: Dual boot, Single boot, Bootable Pen drive, VM

A. Dual Boot

In Dual Boot, Linux and another operating system, such as Windows, are installed on the same computer. When the computer starts, a boot menu appears from which we can select either Linux or Windows. After selecting Linux, we can use its terminal and GCC compiler for C programming.

B. Single Boot

In Single Boot, Linux is installed as the only operating system on the computer. The computer directly starts in Linux whenever it is switched on. C programs can then be written using a text editor and compiled and executed through the Linux terminal.

C. Bootable Pen drive

In this method, a Linux ISO file is used to create a bootable USB/Pen drive. The computer is restarted and configured to boot from the USB drive. Linux can then be started in Live mode, allowing us to use the Linux environment and perform C programming without installing Linux on the computer.

D. Virtual Machine (VM)

A Virtual Machine allows Linux to run inside an existing operating system, such as Windows. Software such as VirtualBox or VMware is installed, a new virtual machine is created, and Linux is installed in it using a Linux ISO file. Linux can then be opened in the VM and used for C programming.

1. Virtual Machine Manager (Hypervisor):

Hypervisor, also called a Virtual Machine Manager (VMM), is software or firmware that creates and manages Virtual Machines (VMs). It allows multiple operating systems to run on the same physical computer by sharing hardware resources such as CPU, RAM, and storage.

A. Type-1 (BareMetal)

A Type-1 Hypervisor (BareMetal) runs directly on the computer's physical hardware without requiring a host operating system. The virtual machines and their operating systems run on top of the hypervisor. Examples: VMware ESXi, Microsoft Hyper-V, Xen.

Type-2 (Hosted)

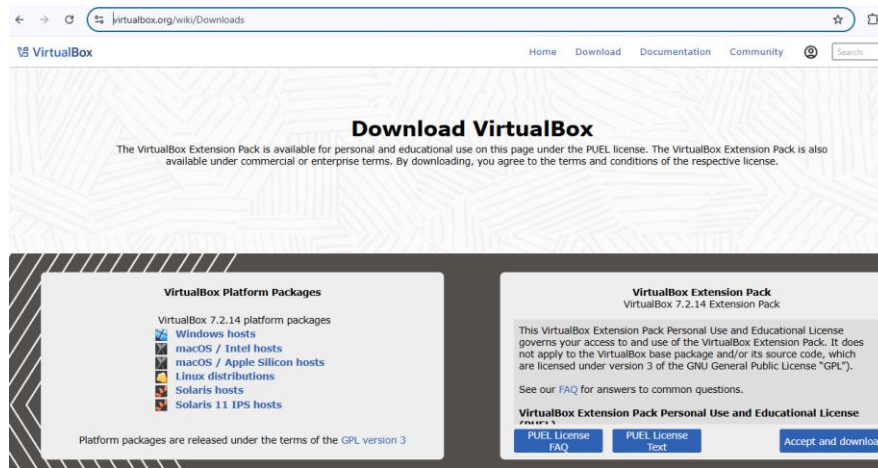
A Type-2 Hypervisor (Hosted) runs as an application on an existing operating system, such as Windows, Linux, or macOS. Virtual machines are created and executed through this application.

Examples: Oracle VirtualBox, VM Ware.

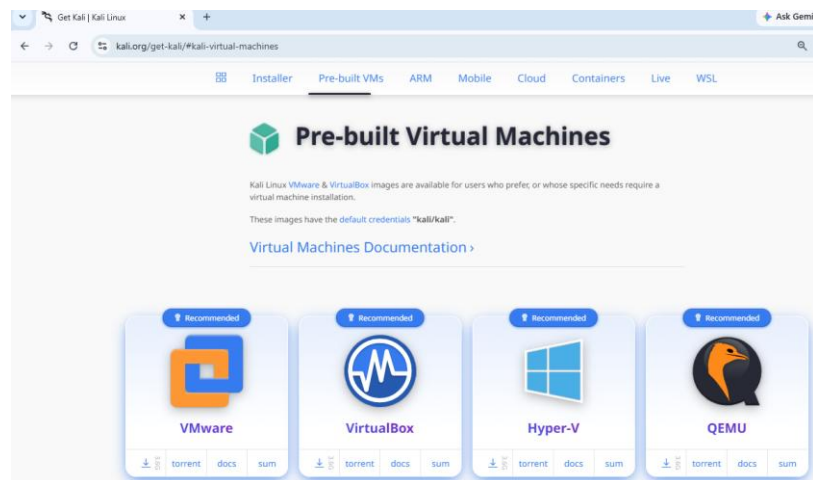
We would be using Oracle VirtualBox.

2. Installation of VMM & VM

Step-1: Download Oracle Virtual Box for “Windows hosts”

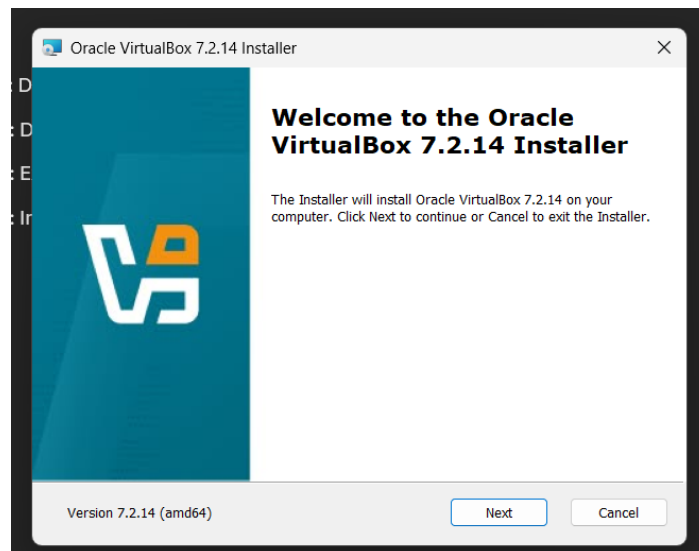


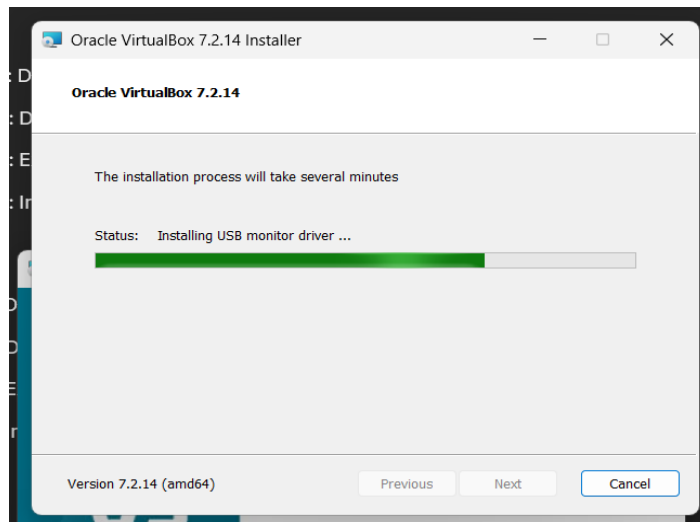
Step-2: Download Pre-built Kali Linux VM for Virtualbox



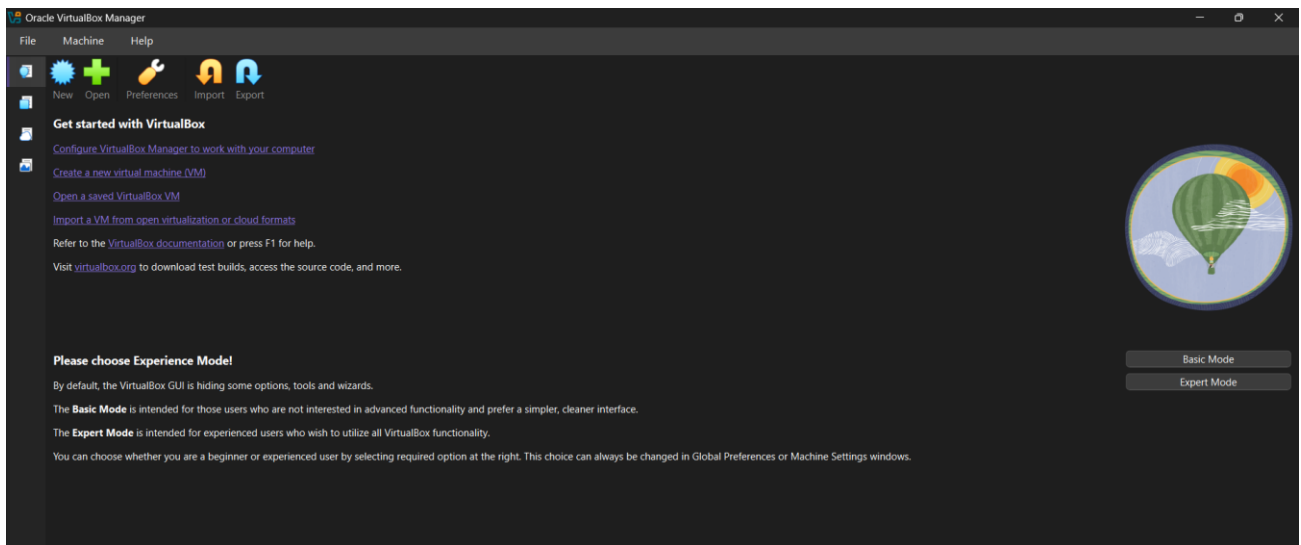
Step-3: Extracted the Kali Linux downloaded file

Step-4: Install Virtualbox

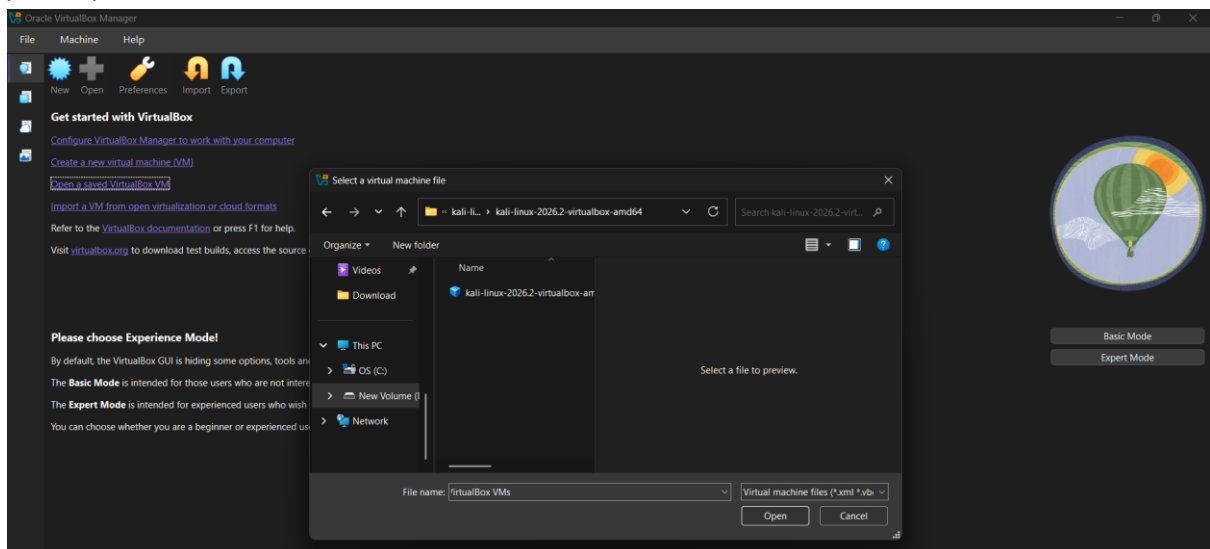


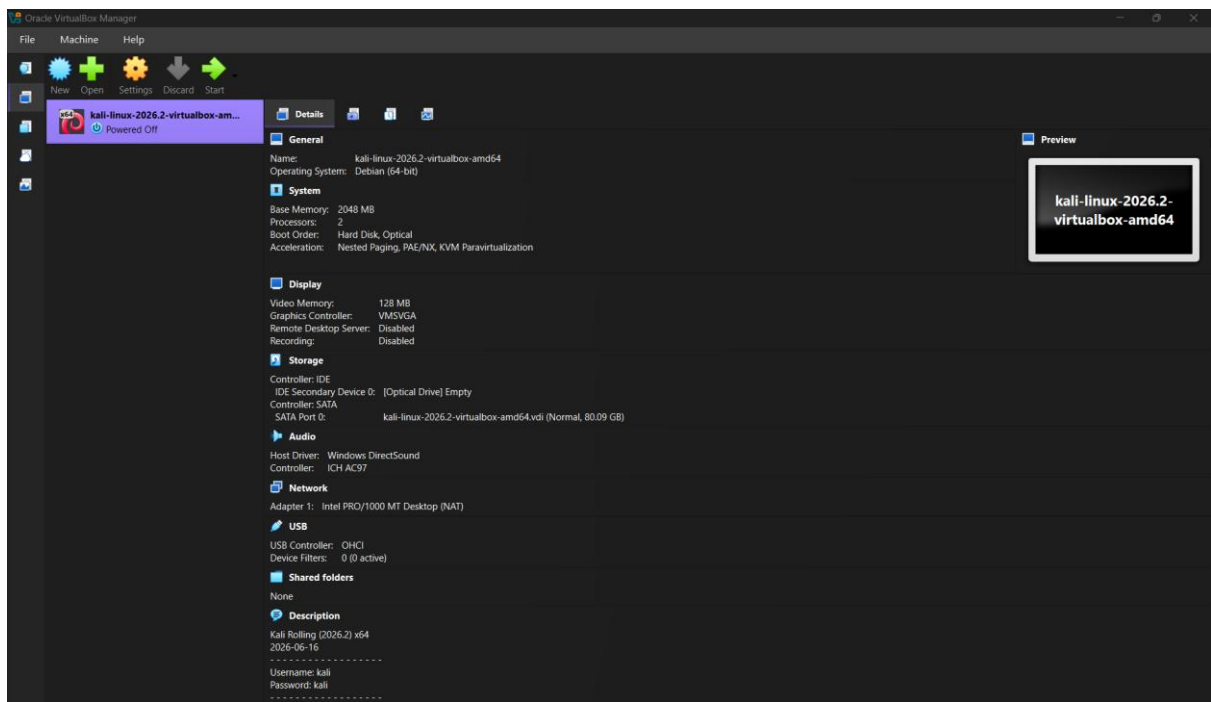


Step-5: Open Virtual box

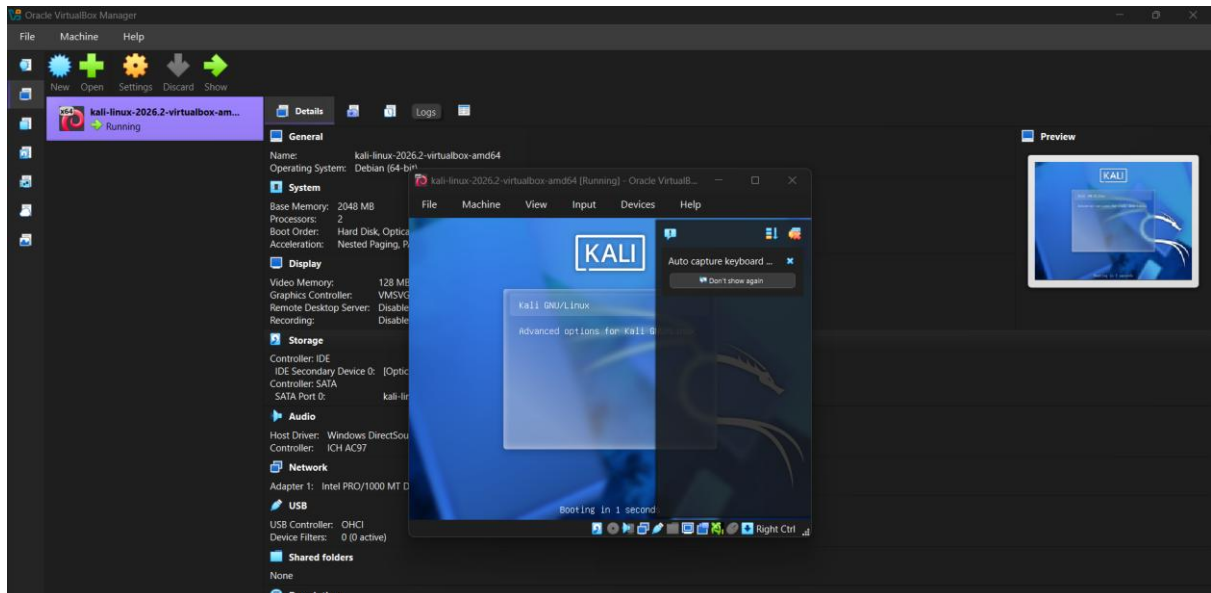


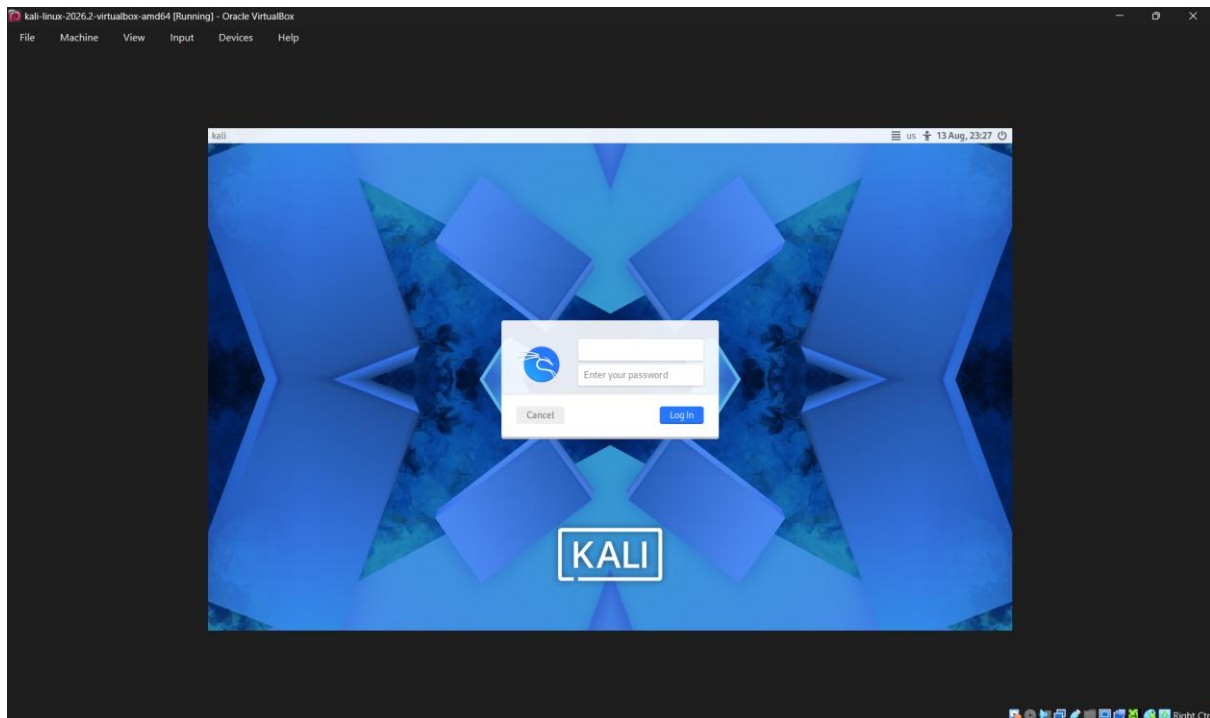
Step-6: Open the “saved VM”





Step-7: Start the VM





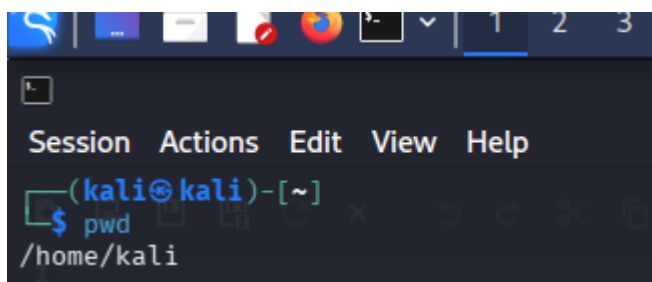
Thus, the installation of VMM (Hypervisor Type-2 Oracle VirtualBox) and VM (Kali Linux) is completed.

Step-8: Open terminal and create the following simple program, compile and execute to understand the working of the environment.

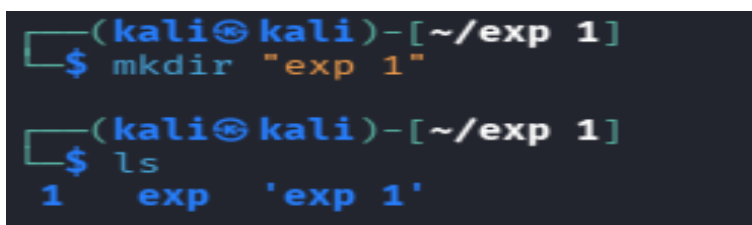
B) Working with Linux Commands

- Terminal basics:

1.pwd: Shows the directory (folder) you are currently in.
e.g: /home/kali



2. mkdir: Creates a new directory called “exp 1”



3. Rmdir: Deletes a directory, but the directory must be empty.

```
(kali㉿kali)-[~/exp 1]
$ mkdir temporary

(kali㉿kali)-[~/exp 1]
$ dir
1 exp exp\ 1 temporary

(kali㉿kali)-[~/exp 1]
$ rmdir temporary

(kali㉿kali)-[~/exp 1]
$ dir
1 exp exp\ 1
```

4. Dir: shows the files and folders present in your current directory.

```
(kali㉿kali)-[~/exp 1]
$ mkdir temporary

(kali㉿kali)-[~/exp 1]
$ dir
1 exp exp\ 1 temporary

(kali㉿kali)-[~/exp 1]
$ rmdir temporary

(kali㉿kali)-[~/exp 1]
$ dir
1 exp exp\ 1
```

5. ls: lists the files and folders in the current directory. It is the commonly used Linux command for listing contents.

```
(kali㉿kali)-[~/exp 1]
$ ls
1 exp 'exp 1'
```

6. cd : Changes your current directory. Here, it takes you inside “exp 1”

```
(kali㉿kali)-[~]
$ cd "exp 1"

(kali㉿kali)-[~/exp 1]
$ pwd
/home/kali/exp 1
```

7. clear: Clears the terminal screen. It does **not** delete files or history.

```
(kali㉿kali)-[~/exp 1]  
$
```

8. history: Displays a list of commands you have previously entered in the terminal.

```
(kali㉿kali)-[~/exp 1]  
$ history  
1  ls  
2  ls  
3  gcc lab 1 circle .c -o circle  
4  gcc lab  
5  gcc "lab 1 circle .c" -o circle  
6  gcc  
7  gcc "lab 1 circle .c" -o circle  
8  ./circle  
9  gcc "lab 1 circle .c" -o circle  
10 ./circle  
11 cc-save-temps lab 1 circle .c.i  
12 cc - save-temps lab 1 circle .i  
13 cc -save-temps lab 1 circle.i  
14 cc -save-temps lab 1 circle .c  
15 gcc -s lab 1 circle .c  
16 gcc -s "lab 1 circle .c" -o "lab 1 circle.s"  
17 cat "lab 1 circle.s"  
18 pwd  
19 mkdir "exp 1"  
20 mkdir "exp 1"  
21 mkdir exp 1  
22 cd exp 1  
23 cd "exp 1"  
24 pwd  
25 mkdir exp 1  
26 ls  
27 mkdir "exp 1"  
28 ls  
29 whoami  
30 mkdir temporary  
31 dir  
32 rmdir temporary  
33 dir
```

9. man: Opens the manual/help page for a command. Here it shows information about ls. Press q to exit.

```
LS(1) User Commands LS(1)
NAME
  ls - list directory contents
SYNOPSIS
  ls [OPTION]... [FILE]...
DESCRIPTION
  List information about the FILES (the current directory by default). Sort entries alphabetically if none of -cftuvSUX nor --sort is specified.
  Mandatory arguments to long options are mandatory for short options too.
  -a, --all
    do not ignore entries starting with .
  -A, --almost-all
    do not list implied . and ..
  --author
    with -l, print the author of each file
  -b, --escape
    print C-style escapes for nongraphic characters
  --block-size=SIZE
    with -l, scale sizes by SIZE when printing them; e.g., '--block-size=M'; see SIZE format below
  -B, --ignore-backups
    do not list implied entries ending with ~
  -c
    with -lt: sort by, and show, ctime (time of last change of file status information); with -l: show ctime and sort by name; otherwise: sort by
    ctime, newest first
  -C
    list entries by columns
  --color[=WHEN]
    color the output WHEN; more info below
  -d, --directory
    list directories themselves, not their contents
Manual page ls(1) line 1/222 17% (press h for help or q to quit)
```

10. whoami: Displays the name of the user currently logged in, such as kali.

```
(kali@kali)-[~/exp 1]
$ whoami
kali
```

- file

1. vi: Used to create or edit a text file in Linux. Example: vi file opens the file named file in the Vi editor.

```
Session Actions Edit View Help
(kali@kali)-[~]
$ cd "exp 1"
(kali@kali)-[~/exp 1]
$ pwd
/home/kali/exp 1
(kali@kali)-[~/exp 1]
$ vi file.txt
(kali@kali)-[~/exp 1]
$ ls
1 exp 'exp 1' file file.txt
```

2. file: Used to identify the type of a file. For example, file file may show that it is an ASCII text file.

```
(kali@kali)-[~/exp 1]
$ file file
file: ASCII text
```

3. rm: Used to remove or delete a file. Example: rm backupfile deletes backupfile.

```
(kali㉿kali)-[~/exp 1]
$ rm backupfile

(kali㉿kali)-[~/exp 1]
$ ls
1  exp  'exp 1'  file  file.txt

(kali㉿kali)-[~/exp 1]
$
```

4. copyfile: Used to **copy a file**. Example: cp file copyfile creates a copy of file named copyfile

```
(kali㉿kali)-[~/exp 1]
$ mv copyfile backupfile

(kali㉿kali)-[~/exp 1]
$ ls
1  backupfile  exp  'exp 1'  file  file.txt
```

5. mv: Used to **move or rename a file**. Example: mv copyfile backupfile renames copyfile to backupfile.

```
(kali㉿kali)-[~/exp 1]
$ mv copyfile backupfile

(kali㉿kali)-[~/exp 1]
$ ls
1  backupfile  exp  'exp 1'  file  file.txt
```

11. cat: Used to **display the complete contents of a file** on the terminal. Example: cat file.

```
(kali㉿kali)-[~/exp 1]
$ cat file
name: soumya choudhary
batch: 15
course: btech cse
```

12. more: Used to **view the contents of a file one screen at a time**. It is useful for reading large files. Example: more file.

```
(kali㉿kali)-[~/exp 1]
$ more file
name: soumya choudhary
batch: 15
course: btech cse
```

13. less: Used to **view and scroll through a file** using the keyboard. It allows movement both up and down. Press q to exit.

```
(kali㉿kali)-[~/exp 1]
$ less file

(kali㉿kali)-[~/exp 1]
$ head -2 file
name: soumya choudhary
batch: 15
```

14. head: Used to **display the beginning of a file**. By default, it shows the first 10 lines. Example: head -2 file shows the first 2 lines.

```
(kali㉿kali)-[~/exp 1]
$ less file

(kali㉿kali)-[~/exp 1]
$ head -2 file
name: soumya choudhary
batch: 15
```

15. tail: Used to **display the end of a file**. By default, it shows the last 10 lines. Example: tail -1 file shows the last 1 line.

```
(kali㉿kali)-[~/exp 1]
$ tail -1 file
course: btech cse
```

- redirection:

1. > : output redirection

sends the output of a command to a file instead of displaying it on the terminal

```
(kali㉿kali)-[~/experiment]
$ cat file1
hello world

(kali㉿kali)-[~/experiment]
$ cat file1 > output

(kali㉿kali)-[~/experiment]
$ ls
file1 output

(kali㉿kali)-[~/experiment]
$ cat output
hello world
```

2. >> : append redirection

also sends output to a file, but it adds the new output at the end instead of replacing the old contents

```
(kali㉿kali)-[~/experiment]
$ cat file1 >> output
hello world
hello world
```

3. <: input redirection

tells a command to take its **input from a file** instead of taking input from the keyboard.

```
(kali㉿kali)-[~/experiment]  
$ cat < file1 output  
hello world
```